

复旦大学计算机科学与技术学院
2024-2025学年第一学期期末试卷

课程名称: Python程序设计 (AI) 课程代码: ATBI10002

卷 别: ☒ A 卷 ☐ B 卷 ☐ C 卷

姓 名: 学 号:

提示: 请同学们秉持诚实守信宗旨, 谨守考试纪律, 摒弃考试作弊。学生如有违反学校考试纪律的行为, 学校将按《复旦大学学生纪律处分条例》规定予以严肃处理。

题号	一			二		三	四			总分
	1	2	3	1	2	1	1	2	3	
得分										

一、阅读程序写输出 (30%)

第1题 (10%)

```
def calculate_price(amount, vip, celebration):  
    if vip:  
        if amount >= 1000:  
            price = amount * 0.8  
        elif amount >= 500:  
            price = amount * 0.9  
        else:  
            price = amount  
    else:  
        if amount >= 1000:  
            price = amount * 0.9  
        elif amount >= 500:  
            price = amount * 0.95  
        else:  
            price = amount  
    if celebration:  
        price = price * 0.9  
    return price
```

测试用例

```
test_cases = [  
    (1000, True, False), # 案例1  
    (1000, True, True), # 案例2  
]
```

print("案例输出结果:")

```
for amount, vip, celebration in test_cases:
```

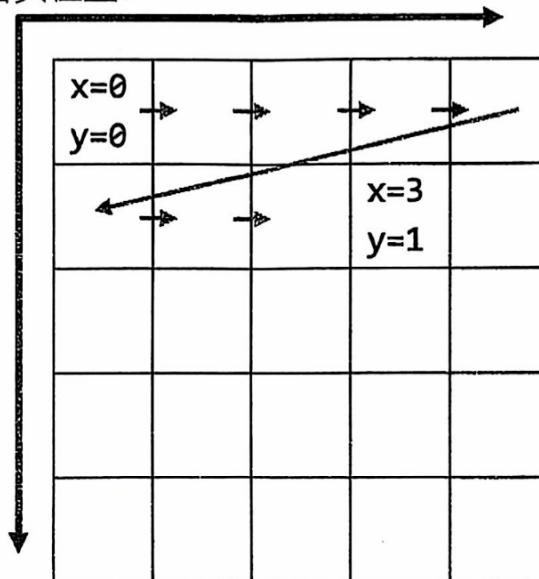
```
    result = calculate_price(amount, vip, celebration)
```

```
    print("消费金额:%d,最终价格:%.1f\n" % (amount,result))
```

#-----

第2题 (10%)

在一局打地鼠游戏中，地鼠窝是一个 5X5 的空间，有 25 个出口。地鼠在当前位置按前进 1 格、2 格交替步伐跳动到下一位置（从左向右，一行到头就从下一行最左边数；到达右下角就回到左上角），地鼠的起始位置为 (x=3, y=1) 请预测地鼠的 5 次露头位置。



```
x, y = 3, 1
```

```
step = [1, 2]
```

```
step_total = y*5+x
```

```
for i in range(5):
```

```
    step_i = step[i%2]
```

```
    step_total = step_total + step_i
```

```
    x = (step_total % 25) % 5
```

```
    y = (step_total % 25) / 5
```

```
    print('( %d,%d)'%(x, y))
```

第3题 (10%)

题目: 给定一个列表, 输出以下切片结果:

```
lst = list(range(10))
```

```
slice1 = lst[:3]
```

```
print(slice1)
```

```
slice2 = lst[-3:]
```

```
print(slice2)
```

```
middle_index = len(lst) // 2
```

```
slice3 = lst[middle_index - 1 : middle_index + 2]
```

```
print(slice3)
```

```
slice4 = lst[::2]
```

```
print(slice4)
```

```
slice5 = lst[1::2]
```

```
print(slice5)
```

二、程序填空 (20%)

第1题 (10%)

```
# -*- coding: utf-8 -*-
```

```
'''-----
```

【程序填空】

```
-----
```

程序实现了多个值的累加和的功能, 比如 1, 2, 3 这 3 个值的累加和为: 1, 1 + 2, 1 + 2 + 3。请补全缺失的部分, 使得该程序运行结果如下所示。

[1, 3, 6]

```
-----
```

注意:

(1) 每行限填一行代码。

(2) 除要求填空的位置之外, 请勿改动程序中的其他内容。

```
-----'''
```

```
total = 0
```

```
def moving_sum(x, reset=False):
```

```
    1
```

```
    if reset:
```

```
        total = 0
```

```
    total += x
```

```
    return total
```

```

    if not values:
        return None
    first = moving_sum(values[0], True)
    results = [first]
    for value in values[1:]:
        current = moving_sum(value)
        results.append(current)
    return results

if __name__ == '__main__':
    print(cumulative_sum(1, 2, 3))

```

第2题 (10%)

注意：

- (1) 每行限填一行代码。
- (2) 除要求填空的位置之外，请勿改动程序中的其他内容。

下列的 Python 程序中有 2 个空格，请根据要求完成程序填空。该程序接收一个英文人名和一个以斤为单位的体重，经处理后输出一条格式化的消息，示例如下：

如果用户输入的名字是 `alice`，体重是 98 斤，则输出：
Hello, ALICE! Your weight is 49.0 kg.

要求：

- 1、对名字字符串正确处理，使得名字中的所有字母大写。
- 2、输出的消息需要确保名字和体重正确输出，体重转换为以公斤为单位，并且使用 `f-string` 将名字和体重插入到输出消息中，且体重保留 1 位小数。

```

# 获取用户输入的名字和体重
name = input("Please enter your name: ")
weight = int(input("Please enter your weight: "))

# 将名字中的所有字母大写，体重转换为以公斤为单位
formatted_name = _____ 1 _____
formatted_weight = weight/2.0

# 使用 f-string 输出消息（注意体重保留 1 位小数）
message = f"Hello, {formatted_name}! Your weight is
{_____ 2 _____} kg."

```

```
# 打印消息
print(message)
```

三、程序改错（10%）

''' _____

【程序改错】
以下程序通过列表推导从嵌套列表中提取出所有长度大于 3 的字符串，并将其变为大写形式存储在新列表中。并检查指定字符串（不区分大小写）是否存在。

以下程序有两处错误，请在答题位置填写错误行的编号（每行最右边注释里的数字。如果代码行后面没有注释，表示该行不会出错）和改正后的整行代码。改正后的代码只能是一行，不允许跨越多行。
_____ , , ,

```
#原始嵌套列表
nested_list=[["apple","cat","banana"],["dog","elephant"],
["fish","grape","hen"]]
check = "elephant"

#提取符合条件的字符串，并转换为大写
results=[word.upper() for sublist in nested_list
          for word if len(word)>3] #1

#打印结果
print("Filtered and Transformed Strings:",results) #2

#检查是否包含特定单词（不区分大小写）
if check in results: #3
    print(f"The word'{check}'exists in the results.") #4
else: #5
    print(f"The word'{check}'does not exist in the results.")#6
```

错误行 编号	改正

四、编程（40%）

第1题（10%）

编程求解一元二次方程 $ax^2 + bx + c = 0$

从键盘读取方程系数 a、b 和 c，根据求根公式 $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ ，计算并输出方程的实数根。

注意，根的个数可能为 0 个、1 个或 2 个。

系数 a、b、c 在同一行从键盘读入，以逗号隔开。假设用户输入的 a，b，c 都是合法的数值，且 a 不等于 0。

第2题（10%）

阅读并完成下面 Python 程序，该程序将处理一个班级的成绩数据，获得这个班级的等级数据并打印输出。该程序的运行效果如下：

成绩等级单如下[每等级的姓名按成绩从高到低排列]：

1. 等级A的同学：

吴傲天, 沈雨, 周胜利

2. 等级B的同学：

韩雪, 钱力, 赵华, 张风, 冯华琼

3. 等级C的同学：

王丽丽, 秦海洋, 李涛

4. 等级D的同学：

褚闪, 孙萌萌

5. 等级F的同学：

王娟, 朱瑜

注意：部分源程序给出如下。请勿修改程序其他部分的内容，仅需编写在 Program，End 注释标志之间的若干语句。

```
def key_func(item):
```

```
    return item[1]
```

```
def get_grade_sheet(class_sheet):
```

```
    """ 获得成绩等级单，采用 dict 来记录等级成绩单，key 为等级：
```

```
    A(>=90), B(>=80 <90), C(>=70 <80), D(>=60 <70), F(<60)
```

```
    value 为相应等级的学生姓名列表，顺序为分数从高到低
```

```
    """
```

```
    grade_sheet = {'A': [], 'B': [], 'C': [], 'D': [], 'F': []}
```

```
    *****Program*****
```

```
    ***** End *****
```

```

def print_grade_sheet(grade_sheet):
    """ 打印成绩等级单，每个等级占两行，学生姓名行\t 缩进，姓名用逗号，分隔
    """
    print('成绩等级单如下[每等级的姓名按成绩从高到低排列]:')
    #*****Program*****

    #***** End *****

def main():
    myClass_sheet = {'张风': 86, '王娟': 51, '孙萌萌': 62, '周胜利': 92, '吴傲天': 96, '钱力': 88, '赵华': 87, '李涛': 72, '王丽丽': 79, '冯华琼': 80, '褚闪': 65, '沈雨': 94, '韩雪': 89, '朱瑜': 39, '秦海洋': 75}

    myGrade_sheet = get_grade_sheet(myClass_sheet)
    print_grade_sheet(myGrade_sheet)

if __name__ == '__main__':
    main()

```

第3题 (20%)

背景说明

在数据分析中，经常需要处理数据文件，完成加载、清理、统计、计算以及可视化操作。以下提供完整的 API 示例，包括常见的数据处理方法。

API 示例

1. 数据加载

```
import pandas as pd

# 加载 CSV 文件
df = pd.read_csv('sales_data.csv')

# 查看前几行数据
print(df.head())

# 检查基本信息
print(df.info())

# 检查缺失值
print(df.isnull().sum())
```

2. 数据清理

```
# 删除重复行
df = df.drop_duplicates()

# 填充缺失值
# 示例: 将缺失的 Sales 填充为 0
df['Sales'] = df['Sales'].fillna(0)

# 删除包含空值的行
df = df.dropna()

# 转换列数据类型 (例如 Sales 应为整数)
df['Sales'] = df['Sales'].astype(int)
```

3. 数据计算与统计

```
# 计算销售额的总和、平均值、最大值和最小值
total_sales = df['Sales'].sum()
average_sales = df['Sales'].mean()
max_sales = df['Sales'].max()
min_sales = df['Sales'].min()

# 打印统计信息
print(f"总销售额: {total_sales}")
```

```
print(f"平均销售额: {average_sales}")
print(f"最大销售额: {max_sales}")
print(f"最小销售额: {min_sales}")

# 按产品分组统计总销售额
product_sales = df.groupby('Product')['Sales'].sum()
print(product_sales)

# 按日期统计每日总销售额
daily_sales = df.groupby('Date')['Sales'].sum()
print(daily_sales)
```

4. 数据排序

```
# 按销售额降序排列
sorted_sales = df.sort_values(by='Sales', ascending=False)

# 显示前 5 行
print(sorted_sales.head())
```

5. 数据筛选

```
# 筛选销售额大于 200 的数据
high_sales = df[df['Sales'] > 200]

# 筛选某产品的数据
product_a_sales = df[df['Product'] == 'Product A']

# 筛选某日期范围内的数据
filtered_sales = df[(df['Date'] >= '2024-01-01') & (df['Date'] <= '2024-01-31')]

# 打印筛选结果
print(high_sales)
print(product_a_sales)
print(filtered_sales)
```

6. 数据可视化

- 折线图: 每日销售额趋势

```
import matplotlib.pyplot as plt
```

```
# 假设已按日期计算每日销售额
daily_sales = df.groupby('Date')['Sales'].sum()

# 绘制折线图
plt.plot(daily_sales.index, daily_sales.values)
plt.title('每日销售额趋势')
plt.xlabel('日期')
plt.ylabel('销售额')
plt.xticks(rotation=45) # 日期旋转方便阅读
plt.tight_layout()
plt.show()
```

- 柱状图：产品销售额分布

```
# 按产品分组统计总销售额
product_sales = df.groupby('Product')['Sales'].sum()

# 绘制柱状图
product_sales.plot(kind='bar', color='skyblue', title='产品销售额分布')
plt.xlabel('产品')
plt.ylabel('销售额')
plt.tight_layout()
plt.show()
```

- 饼图：地区销售额占比

```
# 按地区分组计算总销售额
region_sales = df.groupby('Region')['Sales'].sum()

# 绘制饼图
plt.pie(region_sales, labels=region_sales.index, autopct='%1.1f%%',
startangle=90)
plt.title('不同地区销售额占比')
plt.tight_layout()
plt.show()
```

7. 数据导出

```
# 导出清理后的数据
df.to_csv('cleaned_sales_data.csv', index=False)

# 导出分组统计结果
product_sales.to_csv('product_sales_summary.csv')
```

8. 中文处理

```
import matplotlib
matplotlib.rcParams["font.sans-serif"] = ["SimHei"]
matplotlib.rcParams["font.family"] = "sans-serif"
matplotlib.rcParams["axes.unicode_minus"] = False
```

示例数据文件

文件名: sales_data.csv

Date	Product	Sales	Region	Discount
2024-01-01	Product A	200	North	0.10
2024-01-01	Product B	150	South	0.05
2024-01-02	Product A	300	North	0.15
2024-01-02	Product C	100	East	0.00
2024-01-03	Product B	250	South	0.10

考试须知

1. API 示例中的方法与代码为参考, 请结合题目要求进行适当调整。
2. 答案需逻辑清晰, 避免遗漏步骤。
3. 数据文件结构固定, 确保书写的代码能够适配所提供数据。

实际数据文件如下所示, 数据文件名为 network_traffic.csv, 以 csv 格式保存。每条记录给出了从源 IP (Source IP) 到目的 IP (Destination IP) 的采用相应协议 (Protocol) 的负载流量 (Traffic_MB) 以及相应的状态 (Status)。

Source_IP	Destination_IP	Traffic_MB	Protocol	Status
192.168.1.1	10.0.0.1	500	TCP	Success
192.168.1.2	10.0.0.2	NaN	UDP	Failed
192.168.1.3	10.0.0.3	300	TCP	Success
192.168.1.4	10.0.0.1	200	UDP	Failed
192.168.1.1	10.0.0.2	150	TCP	Success
192.168.1.2	10.0.0.3	100	NaN	Success

题目内容

1. 数据清洗（4分）：

- 编写函数 `load_and_clean_traffic(filepath)`
 - 删除重复行。
 - 填充缺失值：
 - `Traffic_MB` 缺失值填充为 0。
 - `Protocol` 缺失值填充为 `Unknown`。
 - 删除所有 `Status` 为 `Failed` 的行。
 - 保存清理后的数据为 `cleaned_network_traffic.csv`。
 - 返回清理后的 `DataFrame`。

2. 统计分析（4分）：

- 编写函数 `calculate_protocol_traffic(data)`
 - 按 `Protocol` 统计总流量 (`Traffic_MB`)，并输出统计结果。
 - 输出每个源 IP (`Source_IP`) 的总流量，格式如下：
192.168.1.1 的总流量为：650 MB

3. 绘图（4分）：

- 按协议 (`Protocol`) 统计总流量 (`Traffic_MB`)。
- 绘制饼图，显示每种协议占总流量的比例，标题为“协议流量占比”，请妥善处理中文。

4. 高级统计（4分）：

- 编写函数 `filter_high_traffic(data, min_traffic)`
 - 筛选出 `Traffic_MB` 大于给定阈值的所有记录。
 - 按 `Destination_IP` 统计筛选结果的流量 (`Traffic_MB`) 总额，输出如下格式：
目标 IP 10.0.0.1 的流量为：700 MB

5. 数据导出与验证（4分）：

- 将筛选结果保存为 `high_traffic.csv`。